Проектирование системы: «Альфред» — умный ассистент путешествий

1. Цели и границы системы

Что делает система:

1. Отслеживает статус рейсов пользователя через внешние API и push/email-рассылки перевозчиков.
2. При задержке/отмене — уведомляет всех участников поездки (многопользовательский режим).
3. Автоматически подбирает альтернативные рейсы/активности с учётом багажа, виз, таймзон, стыковок.
4. Обновляет общий план путешествия и синхронизирует его у всех участников.
5. Работает при офлайне пользователя (накопление и отложенная доставка).
6. Хранит паспортные данные минимально необходимое время (data minimization).

Вне границ: платёжный процессинг, бронирование отелей/активностей, контент рекомендаций — интеграции.

---

2. Ключевые архитектурные решения

- Событийно-ориентированная архитектура (EDA) + CQRS.
- Очередь событий (Kafka) — статусы рейсов, изменения плана, уведомления.
- Idempotent consumer для внешних API (данные приходят с задержкой и дублируются).
- Saga / Orchestrator для сценария «отмена → подбор альтернатив → обновление плана → уведомление».
- Outbox pattern для надёжной публикации событий.
- Feature flags для постепенного включения автоподбора.
- Zero-trust для доступа к PII, шифрование at-rest/in-transit, токенизация паспортных данных.

---

3. Компоненты системы

3.1. Flight Status Tracker (FST)

- Подписка на статусы рейсов: pull (API перевозчиков/агрегаторов с учётом rate limits) и push (webhooks, парсинг рассылок).
- Нормализация статусов: `SCHEDULED`, `DELAYED`, `CANCELLED`, `GATE_CHANGED`, `DIVERTED`.
- Дедупликация и идемпотентность: ключ `(flight_number, departure_date, source, event_id)`.
- Учёт задержки данных: `source_timestamp` и `received_timestamp`, пометка «устаревшие», повторные проверки.
- Fallback при недоступности API: кэш + парсинг рассылок + ручное подтверждение.

3.2. Trip Plan Service (TPS)

- Хранит общий план: рейсы, отели, трансферы, активности, стыковки.
- Многопользовательский режим: `trip_id` → участники (`user_id`, роль owner/member), права на изменения.
- Версионирование плана (optimistic locking) — защита от конфликтов параллельных обновлений.
- Таймзоны: хранение в UTC + IANA tz, конвертация на клиенте, учёт DST.

3.3. Alternative Engine (AE)

- При `CANCELLED`/`DELAYED` (порог настраиваемый) запускает подбор альтернатив.
- Ограничения: багаж (размер/вес, провоз), визы (гражданство/паспорт, страна назначения/транзита), таймзоны и стыковки (MCT), бюджет, предпочтения, доступность мест.
- Источники: API перевозчиков/агрегаторов, кэш.
- Ранжирование: score = f(цена, время, удобство, надёжность, соответствие ограничениям).
- Объяснимость: почему предложен вариант (багаж 23 кг, без визы, стыковка 2 ч).

3.4. Notification Service (NS)

- Каналы: push, email, SMS, in-app, мессенджеры.
- Офлайн-режим: очередь уведомлений, доставка при появлении связи, «тихие» часы.
- Персонализация каналов по настройкам пользователя.
- Идемпотентность: без дублей при повторных событиях.
- Мультиязычность и локализация времени.

3.5. Trip Update Orchestrator (Saga)

1. FST публикует `FlightCancelled`.
2. Orchestrator создаёт `TripIncident`.
3. AE подбирает альтернативы.
4. TPS обновляет план (draft) и фиксирует изменения.
5. NS уведомляет всех участников с вариантами.
6. Пользователь подтверждает → TPS применяет изменения, NS подтверждает.
7. Нет ответа — эскалация (повтор, альтернативный канал, дедлайн).

Компенсации: если бронирование альтернативы не удалось — откат к предыдущему плану, уведомление.

3.6. PII / Passport Vault

- Паспортные данные — в отдельном защищённом хранилище (Vault/KMS), токенизированы; в сервисах только токен.
- TTL: удаление/анонимизация после окончания поездки + grace period (например, 30 дней).
- Least privilege, аудит каждого обращения.
- Шифрование at-rest/in-transit, маскирование в логах.
- Согласие пользователя и право на удаление (GDPR/152-ФЗ).

3.7. Integration Layer

- API Gateway + Adapter-слой для каждого перевозчика (форматы, лимиты, ошибки).
- Circuit Breaker, retry с backoff, bulkhead — изоляция отказов.
- Кэш статусов и альтернатив с TTL; при недоступности API — деградация на кэш + пометка «данные могут быть неактуальны».
- Вебхуки с проверкой подписи.

3.8. Data & Analytics

- Event store для аудита и аналитики.
- Метрики: задержка обнаружения инцидента, доля успешных автоподборов, время до уведомления, конверсия в подтверждение.
- Персонализация без утечки данных: агрегаты/федеративное обучение, без PII в рекомендательном сервисе.

---

4. Потоки данных (сценарии)

Сценарий 1. Задержка рейса

1. Перевозчик присылает webhook `DELAYED` (или FST обнаруживает при pull).
2. FST нормализует, проверяет идемпотентность, публикует `FlightDelayed`.
3. Orchestrator: если задержка > порога — создаёт инцидент.
4. AE подбирает альтернативы (если задержка критична для стыковки).
5. NS уведомляет участников.
6. Пользователь подтверждает → TPS обновляет план → NS подтверждает всем.

Сценарий 2. Отмена рейса, пользователь офлайн

1. FST → `FlightCancelled`.
2. Orchestrator → AE подбирает 3 варианта.
3. NS ставит уведомления в очередь; push не доставлен (офлайн).
4. При появлении сети — push доставляется; in-app и email дублируют.
5. Пользователь подтверждает → план обновляется.

Сценарий 3. Отказ внешнего API

1. Circuit Breaker размыкается.
2. FST использует кэш + рассылки; помечает данные как «возможно устаревшие».
3. NS уведомляет: «Не удалось получить актуальный статус, проверьте у перевозчика».
4. При восстановлении API — повторная проверка и корректировка.

Сценарий 4. Групповая поездка

1. Один участник получает уведомление и предлагает альтернативу.
2. TPS фиксирует draft, запрашивает подтверждение у owner.
3. После подтверждения — план обновляется у всех, NS уведомляет всех.

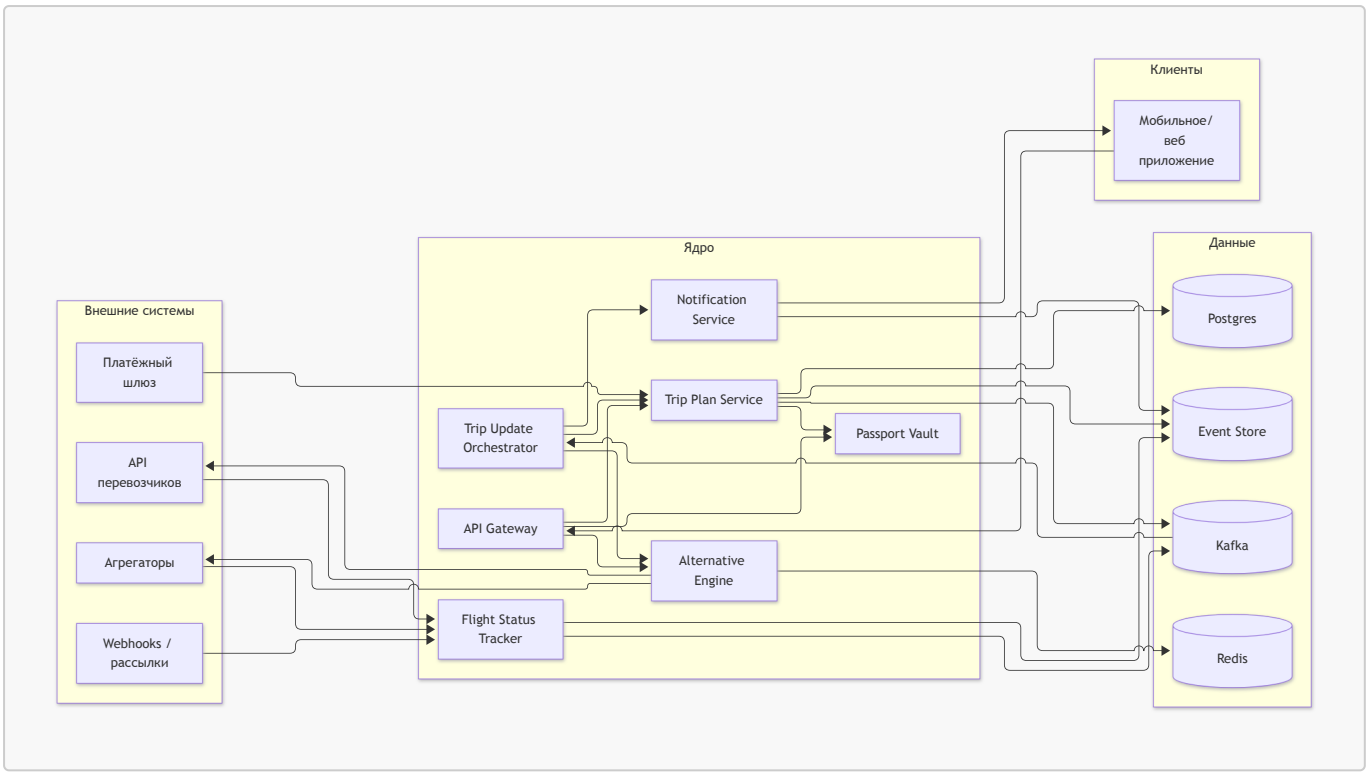
5. Обработка особых требований

Требование	Решение
Данные о рейсах с задержкой	`source_timestamp` vs `received_timestamp`, повторные проверки, дедупликация, пометка «устаревшие»
Пользователь офлайн	Очередь уведомлений, отложенная доставка, дублирование каналов, in-app при возврате
Багаж	Ограничения в AE: размер/вес, провоз, тариф
Визы	Проверка по гражданству/паспорту и стране назначения/транзита; транзитные визы
Таймзоны	UTC + IANA tz, конвертация на клиенте, учёт DST
Паспортные данные	Токенизация, TTL, шифрование, аудит, least privilege
Многопользовательский режим	Роли, версионирование плана, optimistic locking, уведомления всем
Отказы внешних API	Circuit Breaker, retry, кэш, деградация, уведомление о неактуальности

6. Нефункциональные требования

- Доступность: 99.9% для уведомлений, 99.5% для подбора альтернатив.
- Задержка обнаружения: ≤ 5 мин от получения события (при pull — ≤ 15 мин).
- Масштабируемость: горизонтальное масштабирование FST/AE/NS, партиционирование Kafka по `trip_id`.
- Безопасность: OAuth2/OIDC, RBAC, шифрование, аудит, GDPR/152-ФЗ.
- Наблюдаемость: метрики, логи, трассировка (OpenTelemetry), алерты.
- Идемпотентность всех consumer'ов и API.
- Тестируемость: contract tests для внешних API, chaos engineering для отказов.

7. Диаграмма компонентов


Клиенты
Мобильное/ веб приложение
Ядро
Notification Service
Trip Plan Service
Trip Update Orchestrator
Passport Vault
API Gateway
Alternative Engine
Flight Status Tracker
Данные
Postgres
Event Store
Kafka
Redis
Внешние системы
Платёжный шлюз
API перевозчиков
Агрегаторы
Webhooks / рассылки